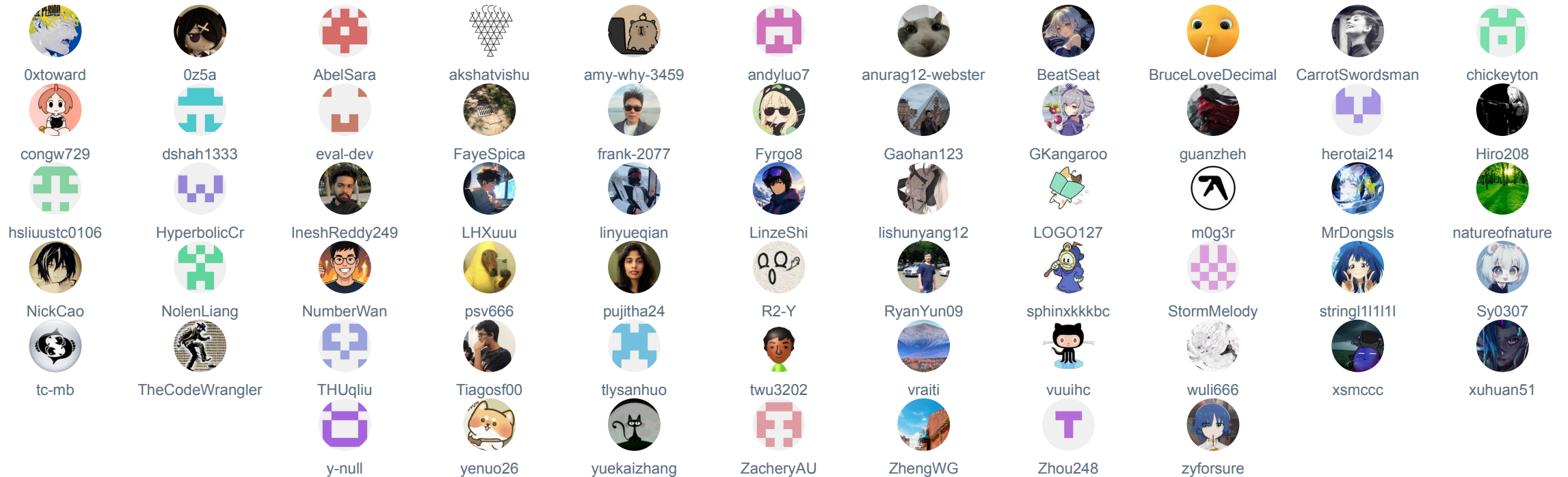


The Next Inflection Point in SI Interaction

How vLLM-Omni Enables Models to
Listen and Speak Simultaneously

Duplex, built with the community

Thanks to 60+ contributors · 160+ related PRs merged

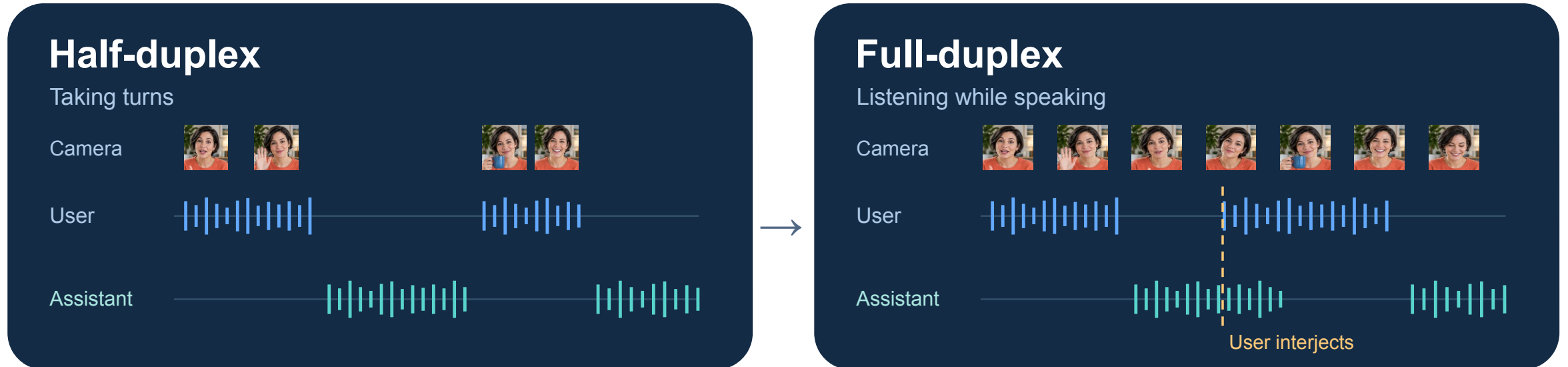


SELECTED MERGED PRs

Native duplex runtime · #3907	Unified duplex framework · #7413	OpenAI Realtime API · #7285	Qwen3-Omni integration · #7585
PersonaPlex unified duplex · #7695	Nemotron VoiceChat · #6089	AURA integration · #7633	Drop cancelled audio · #7644

AS OF 07 OCT 2026

From Half-Duplex to Full-Duplex Interaction



Shared: streamed responses

Both modes can deliver text and audio incrementally.

Concurrent input & output

Keep receiving new input while the assistant responds.

Adaptive interaction

Native duplex models can use new input to shape the next output or yield.

How models handle live interaction

Interaction controls depend on the model and demo profile.

MODEL	INTERACTION	WHAT HAPPENS
PersonaPlex	Continuous speech in 80 ms frames No client commit or Server VAD needed	Listens while generating speech Voice and role stay fixed during the session
MiniCPM-o 4.5	Continuous speech and video input Model decides LISTEN / SPEAK	Listen or respond within the live session No Server VAD required by default
Qwen3-Omni	Server VAD detects user speech Microphone input stays active	Stop queued playback Interrupt the current response*
AURA	Hold to talk; release to submit new audio / video input	Holding stops local playback Server generation can continue

PersonaPlex: explicit cancel restarts conversation context; disconnect requires a new session.

MODEL-DRIVEN SPEECH

PersonaPlex · MiniCPM-o 4.5
The model controls when to speak

LOCAL PLAYBACK STOP

AURA button · Qwen VAD demo
Stops playback; inference may continue

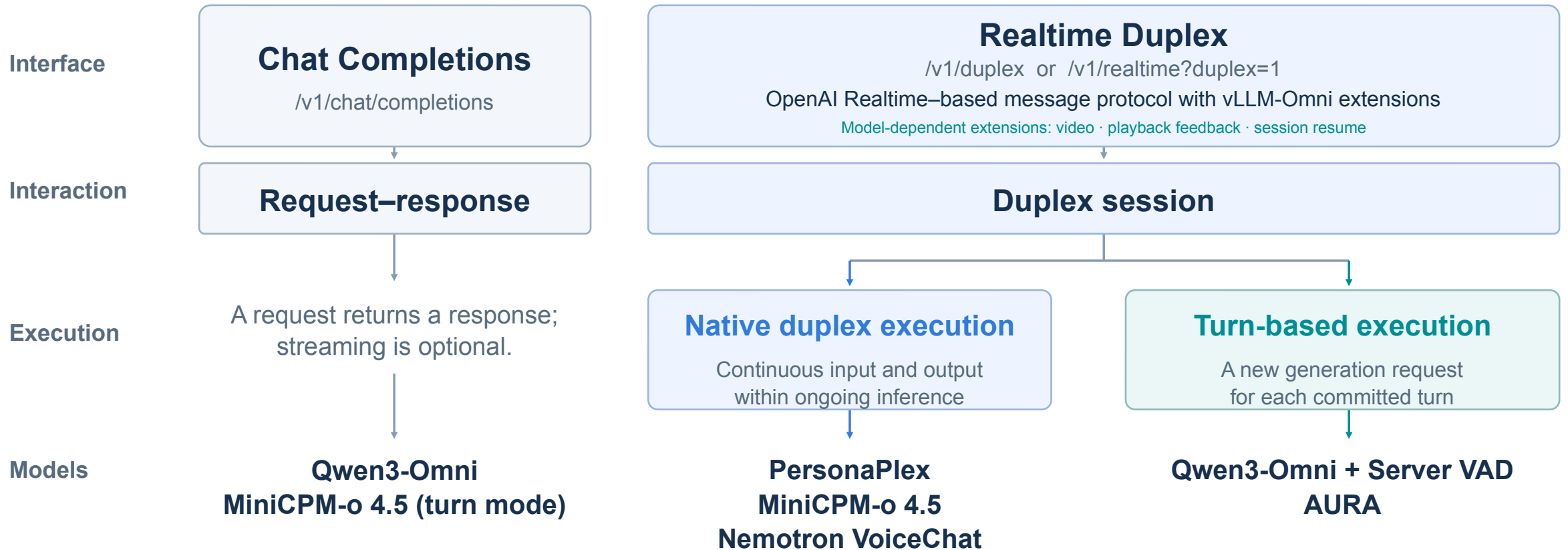
SERVER CANCELLATION

Qwen3-Omni + Server VAD*
Cancels the active generation

* Demo controls vary by model. Qwen VAD cancellation requires interrupt_response.

Interfaces, interaction modes and model execution

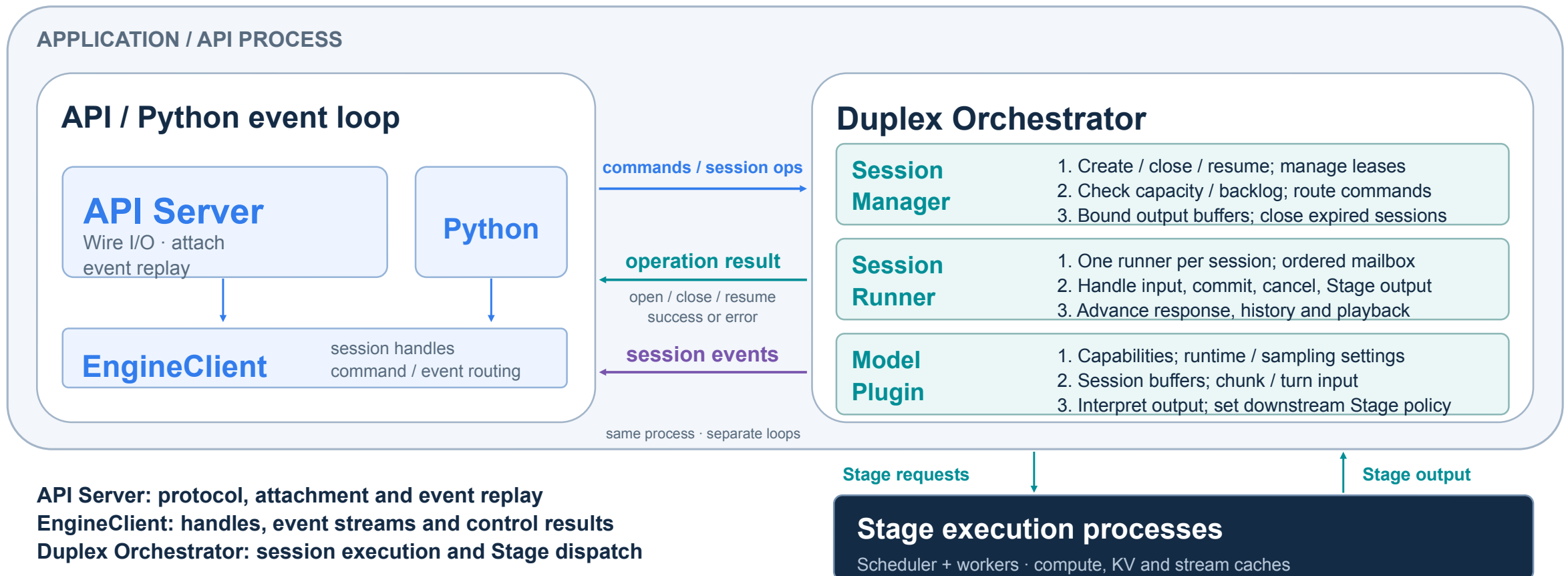
The interface carries the conversation; the execution mode determines how the model processes input.



Realtime also supports a separate turn-based path: /v1/realtime?duplex=0.

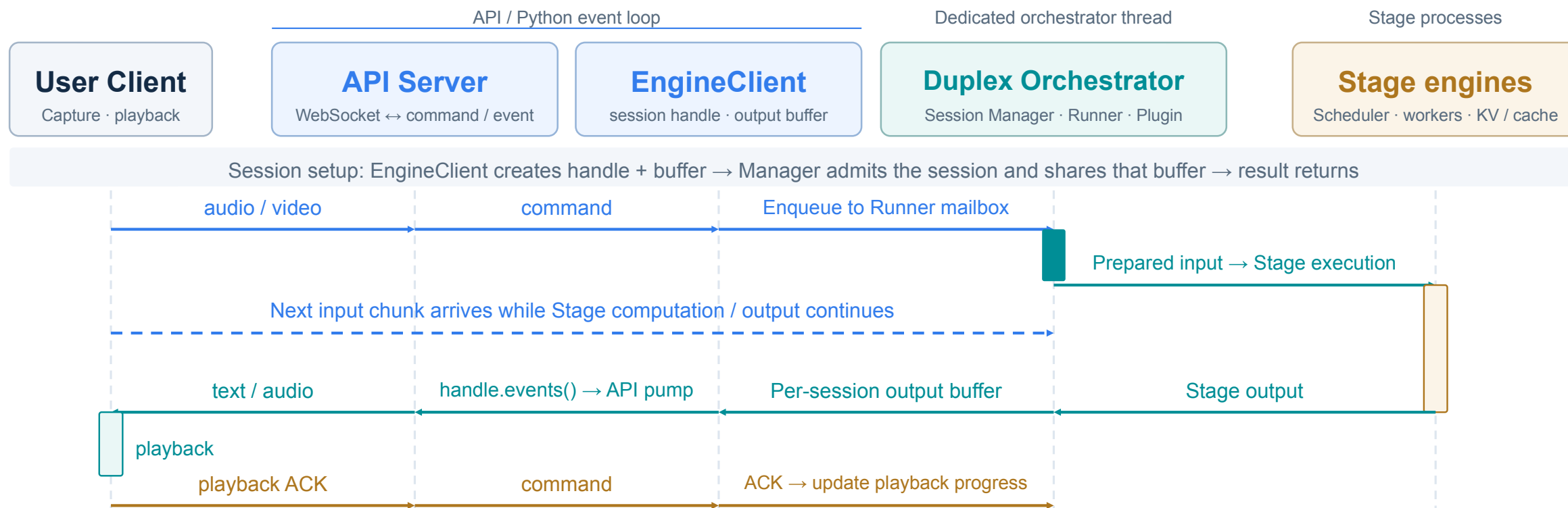
Unified duplex architecture, with clear roles at each layer

Duplex WebSocket and Python sessions share engine-side control, model plugins and bounded output delivery.



Full-duplex workflow

API Server handles protocol; EngineClient routes commands and events; Orchestrator coordinates execution.

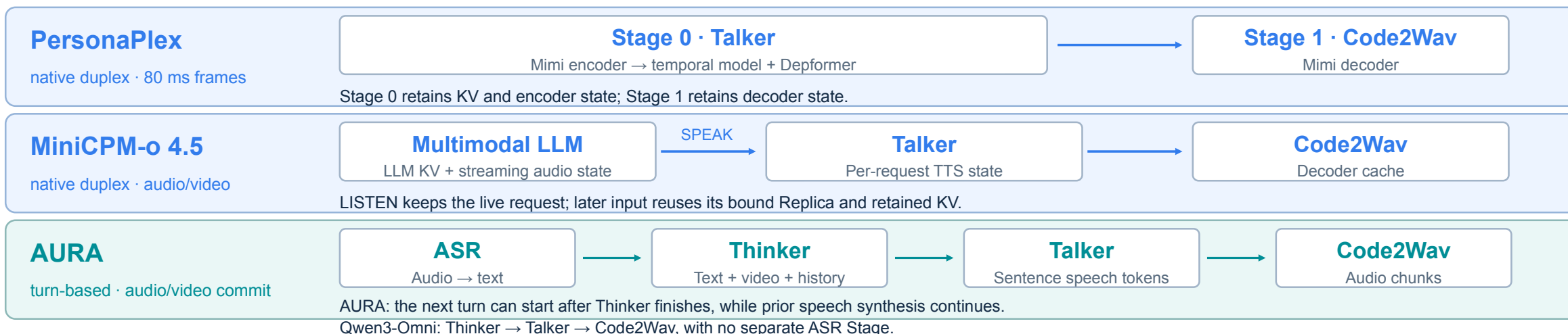


Native: append to a live request · Turn-based: generate after commit · Input can continue in both modes

State handling: live computation and conversation history

PersonaPlex and MiniCPM retain live state; AURA and Qwen3-Omni rebuild input per committed turn.

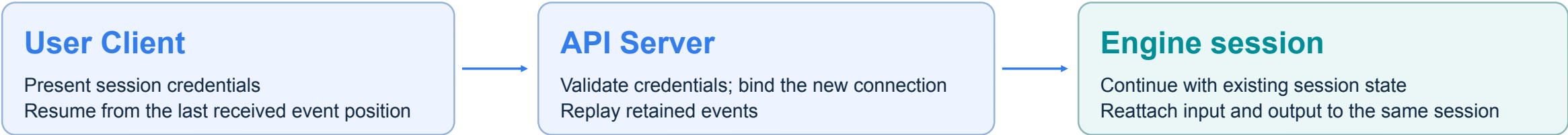
MODEL	INPUT HANDLING	STATE RETAINED ACROSS INPUTS
PersonaPlex	Audio frame → live request	Temporal KV + streaming Mimi encoder / decoder state
MiniCPM-o 4.5	Chunk → continue live request	LLM KV / audio state; configurable sliding window
AURA	Commit turn → new request	ASR text, video clips and assistant text; rebuild prompt each turn
Qwen3-Omni	Commit turn → new request	User audio, image frames and text; rebuild prompt each turn



Stage = one model-compute step; Replica = an execution instance of that Stage.

Connection recovery across models

Session resume: MiniCPM-o 4.5, AURA, Qwen3-Omni and Nemotron VoiceChat.



Reconnect to the same live API process while the session is valid.
Replay requires retained events; process restarts require a new session.

RESUME STEP	FRAMEWORK ACTION	USER CLIENT INTEGRATION
Session identity	Keep the same session	Store session ID and the latest resume token; the Python WebSocket SDK manages both.
Connection binding	Rebind the existing session	Send session.resume first on the new connection; save the new token returned in session.resumed.
Event replay	Replay retained events	Send the last received event sequence. If replay has a gap, start a new session.

PersonaPlex demo: reconnect with a new session and reapply the same voice + persona.

Performance optimizations and steady progress

Four methods reduce repeated work and make better use of the GPU.

CUDA Graphs: reuse prepared execution

Replay GPU work with less CPU launch overhead.

[MiniCPM-o 4.5](#) · [Qwen3-Omni \(images\)](#) · [PersonaPlex](#)

Kernel fusion: combine small operations

Reduce GPU launches and intermediate data movement.

[MiniCPM-o 4.5](#)

Session batching: process streams together

Run new frames from several sessions in one batch.

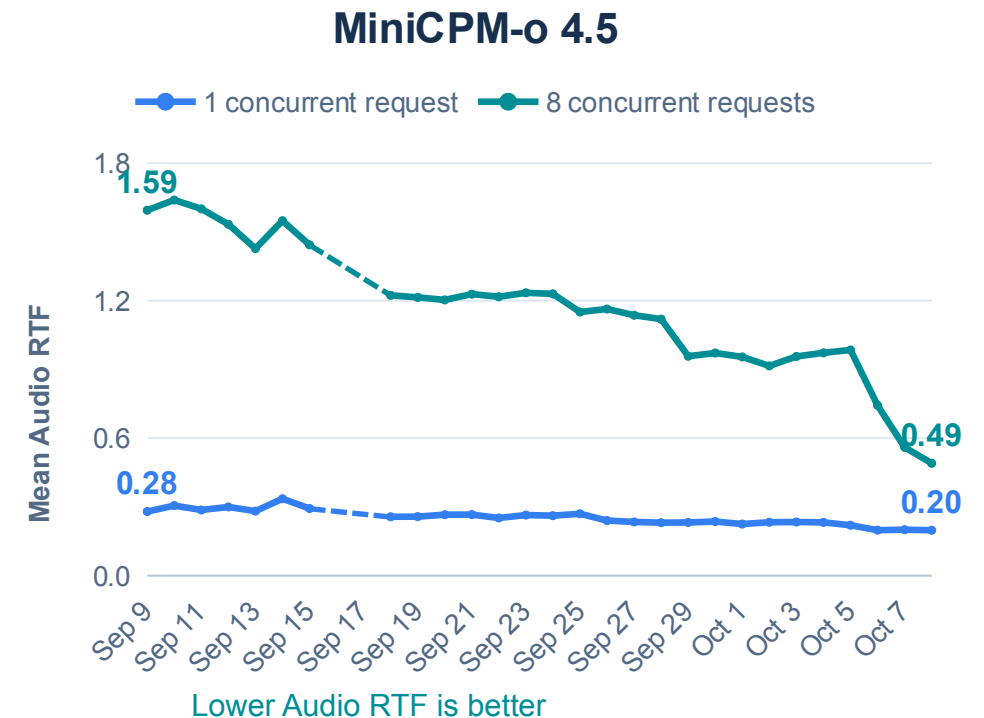
[PersonaPlex](#)

Incremental decoding: handle new audio only

Keep decoder state; decode only new chunks.

[PersonaPlex](#)

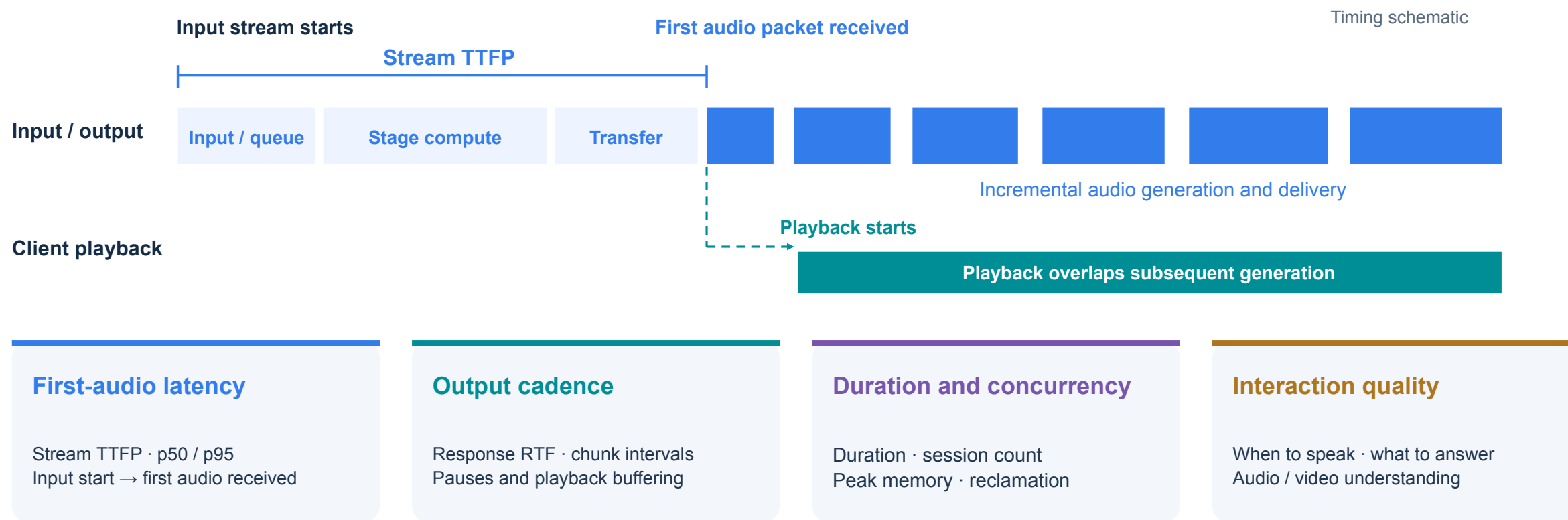
H100 · Chat Completions



Seed-TTS · Sep 9–Oct 8, 2026. Dashed segments bridge dates without matching runs.
Audio RTF = audio generation time / output audio duration, averaged per request.

From first audio to sustained conversation quality

OmnInteract and Omni-DuplexEval evaluate live interaction alongside latency and streaming metrics.



TTFP measures first-packet arrival; conversational response timing is evaluated separately.
Compare matching timing windows. Benchmark coverage varies by model.

Tool calling across duplex models

Existing tool paths provide a foundation for broader support across duplex models.

AVAILABLE TODAY

Qwen3-Omni Realtime

Tool-call parsing · `/v1/realtime?duplex=0`

Nemotron VoiceChat

Tool calls + results in unified native duplex

NEXT: BROADER UNIFIED DUPLEX SUPPORT

Model emits

Tool name + arguments



Application executes

External function



Result returns

Match the `call_id`



Model continues

Generate the reply

During execution

Define whether user input and speech output continue while the tool runs.

Turn safety

Handle interruption, timeouts, duplicate and late results; keep old calls from affecting a new turn.

Framework + plugins

Shared framework handling, with model-specific adapters. Extend support to Qwen3-Omni unified duplex, AURA and JoyAI.

Full-duplex roadmap

Near-term model integration, interaction and evaluation; longer-term session duration and serving scale.

NEAR TERM

JoyAI integration

Bring JoyAI into the unified duplex framework with model-specific input/output adapters

Unified tool calling

Extend tool-call support across models
Handle interruption, timeout and late results

Semantic VAD

Turn completion based on meaning and context
Integration with commit and response policies

Broader realtime evaluation

Broader OmniInteract and Omni-DuplexEval coverage
More models, interaction tasks and regression checks

LONGER TERM

Longer sustained conversations

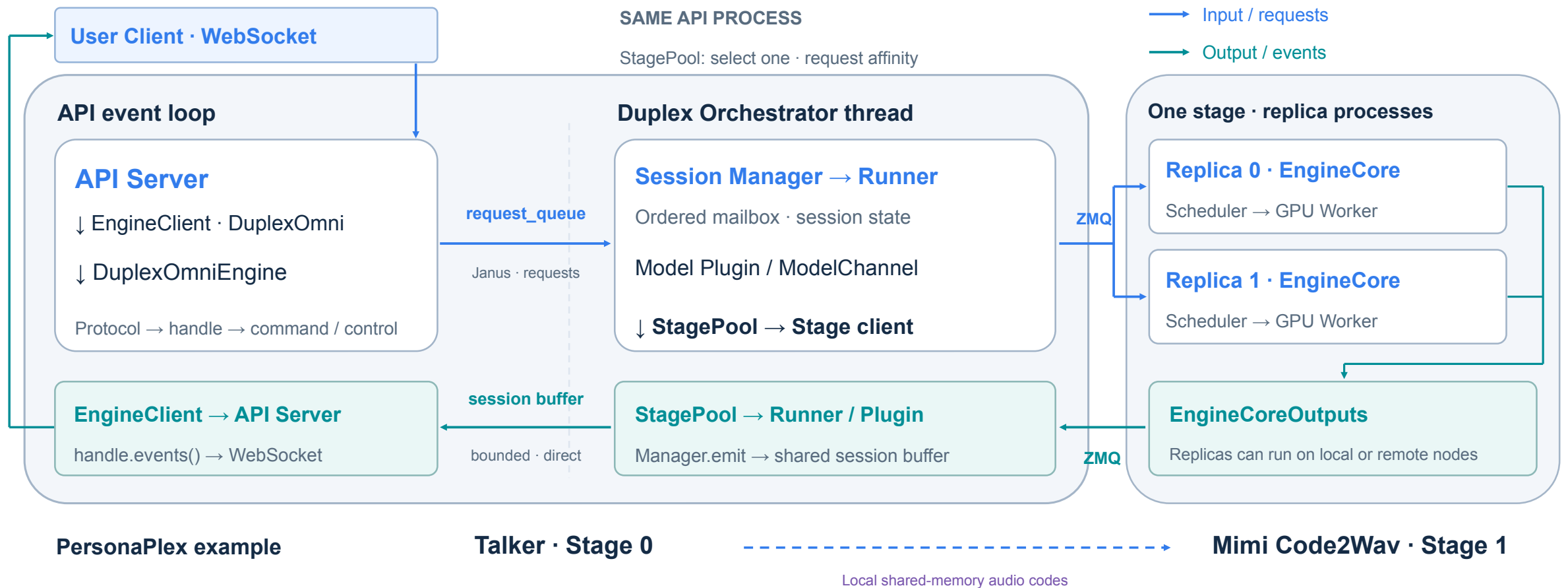
Bound context and history for extended audio/video
Maintain stable latency and memory as sessions grow

Scale beyond one API worker

Build on existing Stage replicas
Route sessions across API frontends

Duplex execution: input, model computation and output

Normal output uses a bounded per-session buffer shared by the API and Orchestrator threads.



Model decisions and VAD interruption

01 Model-driven speech

New input reaches ongoing model execution.
The model chooses whether to keep speaking.

02 VAD-driven turn-taking

Speech onset interrupts the active reply.
Silence triggers commit and a fresh request.

One live conversation

Camera



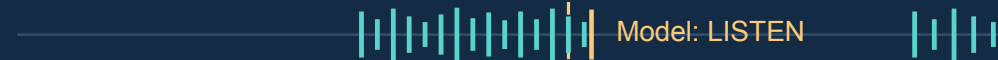
User speech



Barge-in: user speaks

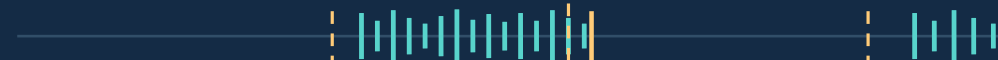


MiniCPM-o 4.5 · Native duplex



Model chooses LISTEN / SPEAK; queued audio can finish.

Qwen3-Omni + VAD · Turn-based



VAD commit VAD-triggered interruption VAD commit

Speech onset stops generation and playback. Silence triggers commit.

Two cancellation entry points

Explicit response.cancel

Client command enters the session cancellation handler.
Core cancellation stops the active generation.

LISTEN is a model decision, separate from these paths.

VAD-triggered interruption

Speech onset activates the overlap policy.
It reuses core cancellation; the client stops playback.

Acoustic VAD vs. Semantic VAD

Often combined: acoustic VAD finds a pause; a turn-end detector checks whether the user has finished.

COMPARISON	Acoustic VAD	Semantic VAD PROPOSED EXTENSION
Decision signal	Speech activity + a configured silence duration	Meaning and context Audio models can also use prosody
Pause handling	May commit after enough silence, even if the sentence is unfinished	Can wait longer when the utterance appears incomplete
System action	May auto-commit after silence. Start a response if enabled.	Commit when the turn is judged complete. Start a response if enabled.
Compute & placement	Low-latency CPU inference Current: CPU thread pool	Small models can run on CPU Heavier models: proposed Omni Stage

OPEN-SOURCE EXAMPLE: SILERO VAD + PIPECAT SMART TURN v3.2

“Book me a flight for tomorrow to ... [pause] ... Beijing.”

1. Silero: speech / non-speech and pauses

2. Smart Turn: complete / incomplete

Server VAD: Runner applies commit / cancel policies; inference runs outside its event loop.

VAD can reduce noise-triggered events. Denoising and interruption intent detection are separate.

Disconnect, cancel and close have different effects

MiniCPM, AURA, Qwen3-Omni and Nemotron support live-session resume; PersonaPlex closes on disconnect.

TRIGGER	IN-FLIGHT WORK	SESSION / RESUME
Disconnect · resumable model	May keep generating; journal later events	Disconnect grace; resume while the session lives
response.cancel	Abort generation; discard unsent output	Session stays open; PersonaPlex resets model context
session.close / lease expiry	Terminate and clean up associated work	Session ends; create a new session

SESSION LIFETIME
Resume within the disconnect grace
Session remains on its live owner

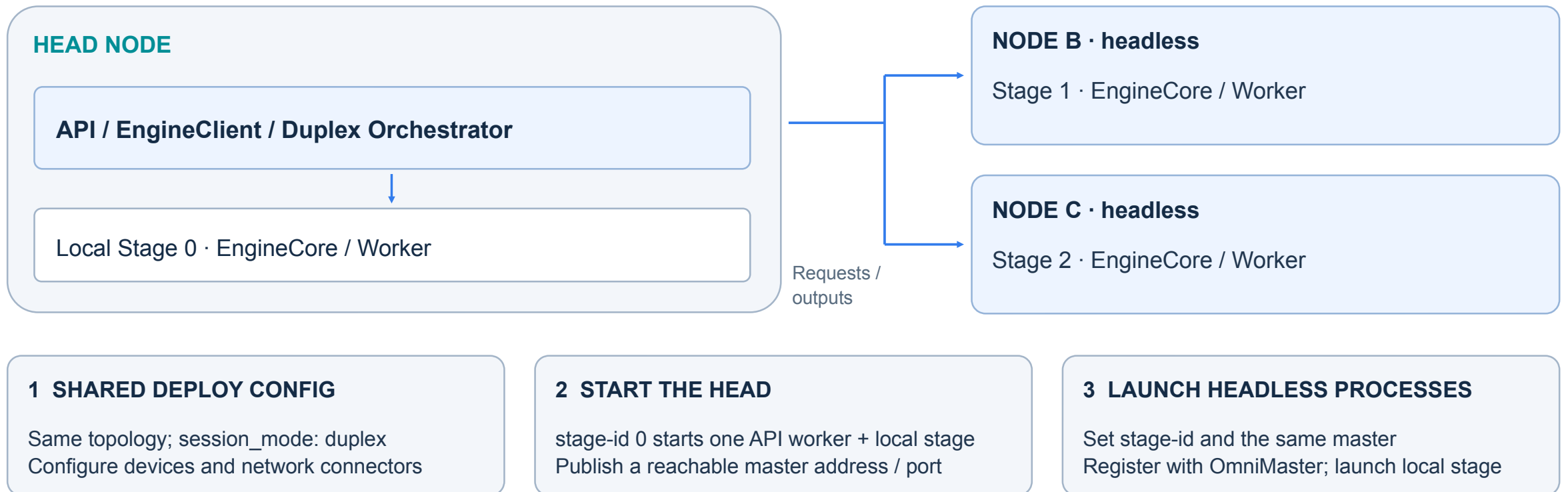
VALID CREDENTIALS
Session ID + latest resume token
Tokens rotate after successful resume

REPLAY COVERAGE
Last received event sequence
A replay gap requires a new session

Defaults: disconnect grace 30s · idle TTL 300s · replay 60s / 8 MiB.
Clients must stop buffered playback separately; resume stays on the same live API process.

Cross-node stages share the entrypoint Orchestrator

One API worker owns duplex sessions; the Head connects local and registered headless Stage processes.



Headless stages share the Head Orchestrator. Duplex rejects multiple API workers; cross-node data needs a network connector.

Code references

LAYER	IMPLEMENTATION ENTRY	READ FOR
API adapter	OmniDuplexSessionHandler	Wire events, attachment, replay
EngineClient	DuplexOmni / DuplexSessionHandle	Handles, shared output buffer, events
Transport	DuplexOmniEngine / DuplexOutputBuffer	Control queues; bounded session output
Session runtime	DuplexSessionManager / DuplexSessionRunner	Leases, state, mailbox, model policy
Stage orchestration	DuplexOrchestrator / StagePool	Initial / update, replica affinity
Model execution	Scheduler / Model Worker	Batching, compute, model caches